

Проект «Чайная Лавка» Полная документация

Интернет-магазин чая на Django

СОДЕРЖАНИЕ

1. Описание проекта
2. Основные возможности
3. Быстрый старт и установка
4. Руководство пользователя
5. Архитектура проекта
6. Структура приложений и моделей
7. Потоки данных
8. Ключевые технические решения
9. Безопасность
10. Масштабирование и production
11. Руководство для разработчиков
12. Лицензия

1. ОПИСАНИЕ ПРОЕКТА

«Чайная Лавка» — полнофункциональный интернет-магазин чая, разработанный на фреймворке Django 4.2+ с использованием Python 3.11+. Проект включает в себя систему авторизации пользователей, корзину покупок, оформление заказов, каталог товаров с поиском и фильтрацией, а также административную панель для управления контентом.

Технологический стек:

- Backend: Django 4.2+
- Database: SQLite (для разработки)
- Frontend: Django Templates + CSS
- Authentication: Встроенная система Django
- Admin Panel: Django Admin с кастомизацией

Статус проекта: учебный, готов к демонстрации и защите.

2. ОСНОВНЫЕ ВОЗМОЖНОСТИ

Система пользователей

- Регистрация и авторизация
- Личный кабинет с редактированием профиля
- История заказов
- Сохранение адреса и телефона

Корзина и заказы

- Добавление товаров в корзину
- Изменение количества товаров
- Удаление товаров из корзины
- Оформление заказа с указанием адреса доставки
- Автоматическое создание заказа и очистка корзины
- История всех заказов пользователя

Каталог товаров

- Категории товаров (Зелёный чай, Улун, Пуэр)
- Детальная страница каждого товара
- Поиск по названию и описанию
- Фильтрация по категориям
- Сортировка по цене и дате добавления
- Отметки «Новинка» и «Хит продаж»

Административная панель

- Управление товарами и категориями
- Просмотр и управление заказами
- Автоматическая генерация slug-полей
- Поиск по названию товаров
- Фильтры по категориям и статусам

Дизайн

- Адаптивная вёрстка для мобильных устройств
- Современный интерфейс с использованием CSS Grid
- Интуитивная навигация
- Красивые карточки товаров

3. БЫСТРЫЙ СТАРТ И УСТАНОВКА

Требования

- Python 3.11 или выше
- Git (для клонирования репозитория)

Установка

Шаг 1. Клонирование репозитория:

```
git clone https://github.com/BPSAY/Tea_Shop_Django.git  
cd tea-shop-django
```

Шаг 2. Создание виртуального окружения:

```
python -m venv venv
```

Шаг 3. Активация виртуального окружения:

Windows:

```
venv\Scripts\activate
```

Linux/Mac:

```
source venv/bin/activate
```

Шаг 4. Установка зависимостей:

```
pip install -r requirements.txt
```

Шаг 5. Применение миграций базы данных:

```
python manage.py makemigrations  
python manage.py migrate
```

Шаг 6. Создание суперпользователя:

```
python manage.py createsuperuser
```

Шаг 7. Запуск сервера разработки:

```
python manage.py runserver
```

Шаг 8. Открытие сайта в браузере:

```
http://127.0.0.1:8000/
```

4. РУКОВОДСТВО ПОЛЬЗОВАТЕЛЯ

Для покупателей

1. Регистрация — создайте аккаунт через кнопку «Регистрация» в верхнем меню.
2. Просмотр каталога — изучите товары, используйте поиск и фильтры для подбора.
3. Добавление в корзину — нажмите кнопку «В корзину» на карточке товара.
4. Управление корзиной — измените количество или удалите товары.
5. Оформление заказа — заполните адрес доставки и подтвердите заказ.
6. История заказов — просмотрите все свои заказы в личном кабинете.

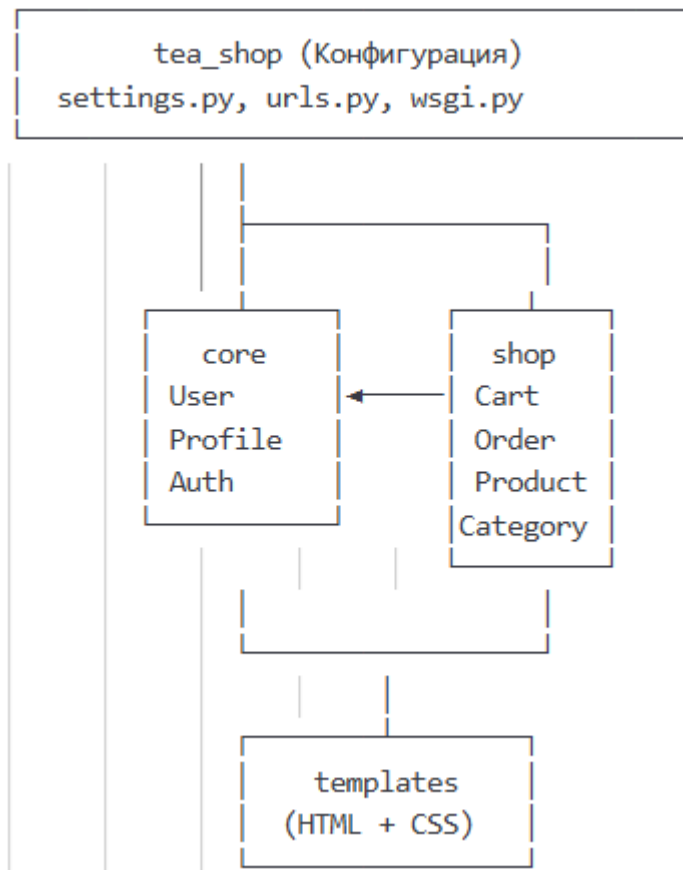
Для администраторов

1. Вход в админку — перейдите по адресу <http://127.0.0.1:8000/admin/>
2. Управление товарами:
 - Добавляйте новые товары с названием, описанием, ценой и фото
 - Назначайте категории
 - Отмечайте новинки и хиты продаж
3. Управление заказами:
 - Просматривайте все заказы
 - Изменяйте статус заказа (Новый → В обработке → Отправлен → Доставлен)
4. Управление пользователями:
 - Просматривайте список зарегистрированных пользователей
 - Редактируйте профили при необходимости

5. АРХИТЕКТУРА ПРОЕКТА

Общая схема

Проект построен по архитектуре MTV (Model-Template-View) с разделением на два независимых приложения: `core` и `shop`.



6. СТРУКТУРА ПРИЛОЖЕНИЙ И МОДЕЛЕЙ

Структура папок

```
tea_shop/
├── core/
│   ├── models.py
│   ├── views.py
│   ├── forms.py
│   └── urls.py
├── shop/
│   ├── models.py
│   ├── views.py
│   └── urls.py
├── templates/
│   ├── base.html
│   ├── core/
│   └── shop/
└── tea_shop/
    ├── settings.py
    └── urls.py
```

Модели данных

Core:

- User (встроенная модель Django) — стандартная модель с полями username, email, password.
- Profile — расширение User через OneToOneField, содержит phone и address. Автоматически создаётся при регистрации через сигналы Django.

Shop:

- Category — категории товаров с автогенерацией slug.
- Product — товары с полями name, slug, category (ForeignKey), description, price, stock, image_url, is_new, is_bestseller.
- Cart — корзина, может быть привязана к User или к session_key.
- CartItem — связь Cart и Product с указанием количества.
- Order — заказ с адресом, телефоном, статусом и итоговой суммой.
- OrderItem — элементы заказа (снимок товара на момент покупки).

Схема базы данных

User (1) $\longleftrightarrow$ (1) Profile
User (1) $\longleftrightarrow$ (1) Cart
Cart (1) $\longleftrightarrow$ (*) CartItem
CartItem (*) $\rightarrow$ (1) Product
Product (*) $\rightarrow$ (1) Category
User (1) $\longleftrightarrow$ (*) Order
Order (1) $\longleftrightarrow$ (*) OrderItem
OrderItem (*) $\rightarrow$ (1) Product

7. БЕЗОПАСНОСТЬ

Реализовано

CSRF protection на всех формах

Валидация входных данных

Защита от SQL-инъекций (Django ORM)

Декоратор `@login_required` для защищённых страниц

Безопасное хранение паролей (хэширование PBKDF2)

Проверка прав доступа к заказам (`user=request.user`)

Рекомендации для production

Установить `DEBUG = False`

Настроить `ALLOWED_HOSTS`

Добавить HTTPS (Let's Encrypt)

Настроить `SECURE_*` настройки Django

Добавить rate limiting

Использовать переменные окружения для `SECRET_KEY`

8. МАСШТАБИРОВАНИЕ И PRODUCTION

Текущие ограничения

1. SQLite — не подходит для production с высокой нагрузкой
2. Нет кэширования — каждый запрос идёт в БД
3. Синхронная обработка — все операции блокируют поток

Рекомендации для production

База данных — PostgreSQL:

```
DATABASES = {  
    'default': {  
        'ENGINE': 'django.db.backends.postgresql',  
        'NAME': 'tea_shop',  
        'USER': 'postgres',  
        'PASSWORD': 'password',  
        'HOST': 'localhost',  
        'PORT': '5432',  
    }  
}
```

Кэширование — Redis:

```
CACHES = {  
    'default': {  
        'BACKEND': 'django_redis.cache.RedisCache',  
        'LOCATION': 'redis://127.0.0.1:6379/1',  
    }  
}
```

Статические файлы — WhiteNoise:

```
STATICFILES_STORAGE = 'whitenoise.storage.CompressedManifestStaticFilesStorage'
```

Асинхронные задачи — Celery:

```
from celery import shared_task  
  
@shared_task  
def send_order_confirmation(order_id):  
    pass
```

Оптимизация запросов

```
products = Product.objects.select_related('category').all()  
categories = Category.objects.prefetch_related('products').all()  
products = Product.objects.only('name', 'price', 'image_url').all()
```


Индексы в БД

```
class Product(models.Model):
    class Meta:
        indexes = [
            models.Index(fields=['category', 'price']),
            models.Index(fields=['slug']),
        ]
```

Варианты хостинга

1. Heroku — простой деплой, бесплатный тариф
2. Railway — современный аналог Heroku
3. PythonAnywhere — специализированный для Python
4. VPS (DigitalOcean, AWS) — полный контроль

Чек-лист перед деплоем

- Установить `DEBUG = False`
- Настроить `ALLOWED_HOSTS`
- Переключиться на PostgreSQL
- Настроить статические файлы
- Настроить media файлы
- Добавить HTTPS
- Настроить email backend
- Добавить логирование
- Настроить резервное копирование БД
- Протестировать все функции

9. РУКОВОДСТВО ДЛЯ РАЗРАБОТЧИКОВ

Сообщения об ошибках

При обнаружении ошибки создайте Issue со следующей информацией:

- Описание ошибки
- Шаги для воспроизведения
- Ожидаемое поведение
- Скриншоты (если применимо)
- Версия Python и Django

Предложения по улучшению

Создайте Issue с описанием:

- Новой функции
- Объяснением полезности
- Возможными вариантами реализации

Pull Requests

1. Форкните репозиторий
2. Создайте ветку: `git checkout -b feature/amazing-feature`
3. Внесите изменения
4. Добавьте тесты (если применимо)
5. Закоммитьте: `git commit -m 'Add amazing feature'`
6. Запушьте: `git push origin feature/amazing-feature`
7. Откройте Pull Request

Стиль кода

- Следуйте PEP 8
- Используйте осмысленные имена переменных
- Добавляйте docstring к функциям и классам
- Комментируйте сложный код

Правила коммитов

Хорошие примеры:

Add shopping cart functionality
Fix user registration validation
Update product filtering logic

Плохие примеры:

Update code
Fix bug
Changes

Планы по развитию

- Интеграция платёжного шлюза (ЮKassa, Stripe)
- Email-уведомления о заказах
- Система отзывов и рейтингов
- Купоны и скидки
- Список желаемого (Wishlist)
- Рекомендательная система
- Мультиязычность (i18n)
- REST API (Django REST Framework)
- PWA (Progressive Web App)

Мониторинг в production

- Sentry — отслеживание ошибок в реальном времени
- Django Debug Toolbar — отладка в development
- Google Analytics — аналитика посещений
- UptimeRobot — мониторинг доступности

10. ЛИЦЕНЗИЯ

MIT License

Copyright (c) 2026 Чайная Лавка BPSAY

Permission is hereby granted, free of charge, to any person obtaining a copy of this software and associated documentation files (the "Software"), to deal in the Software without restriction, including without limitation the rights to use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies of the Software, and to permit persons to whom the Software is furnished to do so, subject to the following conditions:

The above copyright notice and this permission notice shall be included in all copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.

КОНТАКТЫ

- Автор: Артем Леханов
- Email: Dark_rev99@mail.ru
- GitHub: github.com/BPSAY